Lab Sheet 06

Title: Aggregation pipeline operator in MongoDB.

Objective:

- Getting Started with Aggregation Framework.
- Understanding the Group Stage.

What is Aggregation Pipeline in MongoDB?

Aggregation operations group values from multiple documents together and perform a variety of operations on the grouped data to return a single result. The aggregation pipeline refers to a specific flow of operations that processes, transforms, and returns results. In a pipeline, successive operations are informed by the previous result.

Typical Pipeline Flow:

Input → $match → $group → $sort → Output

Stage 1: $match (Filtering)

- It **filters** documents based on conditions
- Only documents that match the condition pass through - Syntax:

  { $match: { fieldName: value } }

Stage 2: $group (Grouping & Aggregating)

- Groups documents by a specific field
- Performs calculations (sum, count, average, etc.) - Syntax:

  { $group: { _id: "$fieldToGroupBy", totalAmount: { $sum: "$fieldToSum" } } }

Stage 3: $sort (Sorting)

- Sorts the results in order (ascending or descending) - Syntax:

  { $sort: { fieldName: 1 } }

  (1 = ascending, -1 = descending)

Database: Student1DB

Collection: student

Student_id	Class	Section	Course_fee
P0001	11	A	200
P0002	10	A	800
P0003	11	A	100
P0004	08	B	300
P0005	11	A	500
P0006	08	B	200
P0007	10	A	400
P0008	11	A	300
P0009	07	B	250
P0010	06	A	100

```
//Create & Switch to Database
use Student1DB

//Create Collection
db.createCollection("student")

//Insert Data into student Collection
db.student.insertMany([
  { Student_id: "P0001", Class: 11, Section: "A", Course_fee: 200 },
  { Student_id: "P0002", Class: 10, Section: "A", Course_fee: 800 },
  { Student_id: "P0003", Class: 11, Section: "A", Course_fee: 100 },
  { Student_id: "P0004", Class: 8,  Section: "B", Course_fee: 300 },
  { Student_id: "P0005", Class: 11, Section: "A", Course_fee: 500 },
  { Student_id: "P0006", Class: 8,  Section: "B", Course_fee: 200 },
  { Student_id: "P0007", Class: 10, Section: "A", Course_fee: 400 },
  { Student_id: "P0008", Class: 11, Section: "A", Course_fee: 300 },
  { Student_id: "P0009", Class: 7,  Section: "B", Course_fee: 250 },
  { Student_id: "P0010", Class: 6,  Section: "A", Course_fee: 100 }
])

//Verify Data
db.student.find().pretty()
```

01. Retrieve all students who are in **Class 11**.

```
db.student.aggregate([
  { $match: { Class: 11 } }
])
```

02. Find the **total course fee** of students in **Section B**.

```
db.student.aggregate([
  { $match: { Section: "B" } },
  { $group: { _id: null, totalFee: { $sum: "$Course_fee" } } }
])
```

03. Calculate the **total course fee for each Class**.

```
db.student.aggregate([
  { $group: { _id: "$Class", totalFee: { $sum: "$Course_fee" } } }
])
```

04. Find the **total course fee for each Section** where course fee is **greater than 200**.

```
db.student.aggregate([
  { $match: { Course_fee: { $gt: 200 } } },
  { $group: { _id: "$Section", totalFee: { $sum: "$Course_fee" } } }
])
```

05. Find the **total course fee for each Class** in **Section A**.

```
db.student.aggregate([
  { $match: { Section: "A" } },
  { $group: { _id: "$Class", totalFee: { $sum: "$Course_fee" } } }
])
```

06. Find the **total course fee for each Class in Section A** and **sort by total fee (descending)**.

```
db.student.aggregate([
  { $match: { Section: "A" } },
  { $group: { _id: "$Class", totalFee: { $sum: "$Course_fee" } } },
  { $sort: { totalFee: -1 } }
])
```

07. Find the **top 2 classes with highest total course fee**.

```
db.student.aggregate([
  { $group: { _id: "$Class", totalFee: { $sum: "$Course_fee" } } },
  { $sort: { totalFee: -1 } },
  { $limit: 2 }
])
```

Exercise

Database: pizza**, collection:** order**.**

Order_id	Pizza_name	Size	Quantity	Order_date
O001	Margherita	Medium	2	2021-06-15
O002	Pepperoni	Large	1	2021-08-20
O003	BBQ Chicken	Medium	3	2022-01-12
O004	Veggie	Small	5	2020-11-10
O005	Margherita	Medium	2	2020-12-02
O006	Pepperoni	Medium	4	2021-06-18
O007	Hawaiian	Medium	1	2021-07-22
O008	BBQ Chicken	Large	3	2022-01-05
O009	Margherita	Medium	3	2022-02-01
O010	Veggie	Medium	2	2021-10-15

```
//Create Database & Collection
use pizza

db.createCollection("order")

//Insert Data
db.order.insertMany([
  { Order_id: "O001", Pizza_name: "Margherita", Size: "Medium", Quantity: 2, Order_date: new
Date("2021-06-15") },
  { Order_id: "O002", Pizza_name: "Pepperoni", Size: "Large", Quantity: 1, Order_date: new
Date("2021-08-20") },
  { Order_id: "O003", Pizza_name: "BBQ Chicken", Size: "Medium", Quantity: 3, Order_date: new
Date("2022-01-12") },
  { Order_id: "O004", Pizza_name: "Veggie", Size: "Small", Quantity: 5, Order_date: new
Date("2020-11-10") },
  { Order_id: "O005", Pizza_name: "Margherita", Size: "Medium", Quantity: 2, Order_date: new
Date("2020-12-02") },
  { Order_id: "O006", Pizza_name: "Pepperoni", Size: "Medium", Quantity: 4, Order_date: new
Date("2021-06-18") },
  { Order_id: "O007", Pizza_name: "Hawaiian", Size: "Medium", Quantity: 1, Order_date: new
Date("2021-07-22") },
  { Order_id: "O008", Pizza_name: "BBQ Chicken", Size: "Large", Quantity: 3, Order_date: new
Date("2022-01-05") },
  { Order_id: "O009", Pizza_name: "Margherita", Size: "Medium", Quantity: 3, Order_date: new
Date("2022-02-01") },
```

```
  { Order_id: "O010", Pizza_name: "Veggie", Size: "Medium", Quantity: 2, Order_date: new
Date("2021-10-15") }
])
```

Tasks:

1. Filter the pizza order documents to pizzas with a size of medium.

```
db.order.aggregate([
  { $match: { Size: "Medium" } }
])
```

2. Returns the total order quantity of medium size pizza grouped by pizza name.

```
db.order.aggregate([
  { $match: { Size: "Medium" } },
  { $group: { _id: "$Pizza_name", totalQuantity: { $sum: "$Quantity" } } }
])
```

3. Filter pizza order documents by date range (2020-01-30 – 2022-01-03).

```
db.order.aggregate([
  {
    $match: {
      Order_date: {
        $gte: new Date("2020-01-30"),
        $lte: new Date("2022-01-03")
      }
    }
  }
])
```

4. Display the total quantity of each pizza ordered, grouped by pizza name.

```
db.order.aggregate([
  { $group: { _id: "$Pizza_name", totalQuantity: { $sum: "$Quantity" } } }
])
```

5. Find the total quantity of medium and large pizzas ordered separately.

```
db.order.aggregate([
  { $match: { Size: { $in: ["Medium", "Large"] } } },
  { $group: { _id: "$Size", totalQuantity: { $sum: "$Quantity" } } }
])
```

6. Find the pizza that has been ordered the most (the one with the highest total quantity).

```
db.order.aggregate([
  { $group: { _id: "$Pizza_name", totalQuantity: { $sum: "$Quantity" } } },
  { $sort: { totalQuantity: -1 } },
  { $limit: 1 }
])
```

7. Find the total quantity of pizzas ordered from 2021-01-01 to 2021-12-31.

```
db.order.aggregate([
  {
    $match: {
      Order_date: {
        $gte: new Date("2021-01-01"),
        $lte: new Date("2021-12-31")
      }
    }
  },
  { $group: { _id: null, totalQuantity: { $sum: "$Quantity" } } }
])
```